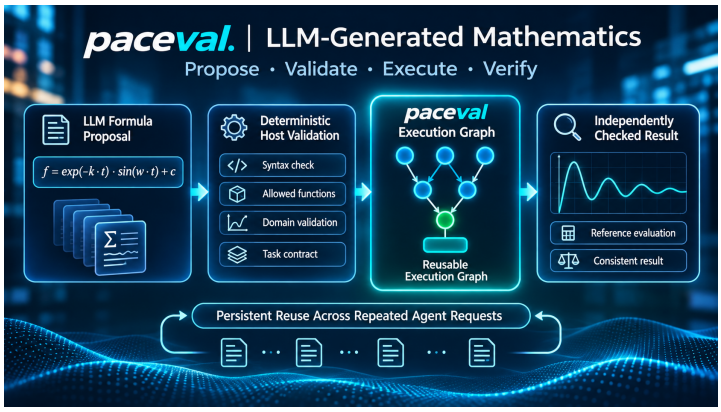
SCIENTIFIC WHITE PAPER · INDEPENDENTLY REPRODUCIBLE C++ DEMONSTRATOR

Validated Mathematical Execution for LLM Tool Proposals with paceval

A tested mathematical tool component for structured agent proposals, versioned model contracts, persistent paceval execution graphs, and independent numerical audits.



Illustrated tool workflow: formula proposal, deterministic validation, reusable paceval execution, and independent mathematical checking.

200	2,560	40/40	30/30
synthetic formula cases	accepted numerical evaluations	deliberately invalid cases rejected	sampled TINC references enclosed

Abstract

The demonstrator combines a bounded C++ untrusted-expression parser, versioned agent model contracts, persistent computation-handle reuse, an owner-approved long-expression path, and paceval mathematical evaluation. A deterministic corpus is screened before vendor execution; valid expressions are evaluated across 16 specified input sets and independently checked by a separate C++ interpreter and 110-digit Python arithmetic. All 2,560 untrusted-path evaluations and 16 additional approved-model evaluations pass their specified tolerances; all 30 sampled TINC intervals include the independent reference.

> **Demonstrated:** two separately verified integration paths: restricted untrusted LLM proposals and a reviewed 1,313-character formula with 48 variables, both executed by the original paceval runtime. A live LLM provider or production MCP transport can be attached in a subsequent integration.

Prepared for paceval UG · Munich, Germany · 11 November 2026. Based on the supplied research proposal “WP_003: Evaluating LLM-Generated Mathematical Expressions.” All numerical findings in this document refer to the included synthetic dataset and its independently executed checks.

01 / SCIENTIFIC PURPOSE AND SCOPE

From a research proposal to an executable mathematical component

The starting proposal identified expression evaluation, persistent computation objects, adversarial formulas and high-precision checking as the research boundary. This implementation turns that proposal into an offline C++ tool component. Its purpose is to **validate, execute and verify proposed mathematical expressions** while checking that requests use an approved engineering model and declared units.

Original proposal

200 expressions, adversarial syntax and singularities, parameter sweeps, and independent numerical verification. The illustrative function was `exp(-k*t)*sin(w*t)+c`.

Implemented extension

An untrusted six-variable AST parser, 10 expression families, sampled TINC, two versioned LLM tool contracts, persistent graph reuse, and a separately approved 48-variable mathematical model.

Offline expression-validation boundary

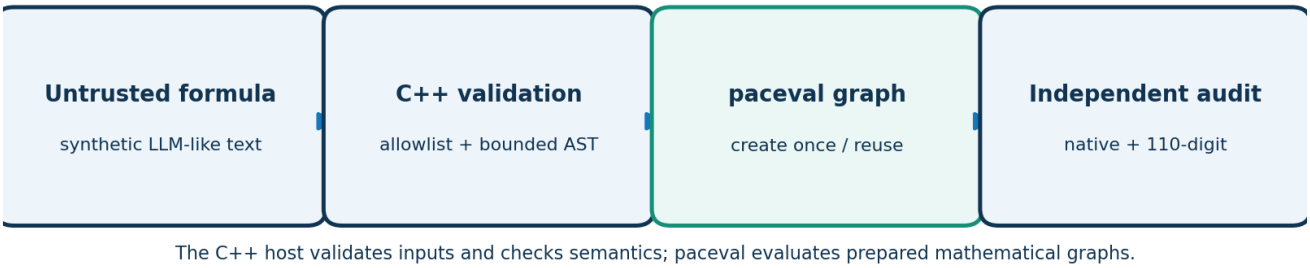


Figure 1. Execution architecture used by the demonstrator. The host validates mathematical syntax and numeric domains before invoking paceval.

The trusted boundary

Formula text from an LLM or agent must be regarded as untrusted. The host implementation accepts only a narrow mathematical language; it does not interpret shell commands, embedded source code or external program calls. The original paceval API receives only expressions which pass this validation. The independent mathematical interpreter can then compare the outputs against a second implementation.

The Model Context Protocol tools specification defines structured tool schemas and requires input validation by servers. OWASP recommends treating model-generated outputs as untrusted and enforcing execution permissions in downstream application code. This demonstrator follows those architectural principles, without claiming to be a complete secure MCP deployment.

This demonstrator keeps formula approval in the host application. Production integrations add authentication, isolated execution and resource quotas appropriate to their environment.

02 / MATHEMATICAL ENGINE AND HOST POLICY

paceval scale is separate from the LLM input restrictions

The official paceval documentation does not impose a fixed expression-length limit or a fixed number of variables. The achievable size depends on available memory, system resources, compiler/runtime environment and supported mathematical functions. The **180-character, 80-node and six-variable values below are choices in the untrusted-input parser of this research demonstrator**, not mathematical-engine limits.

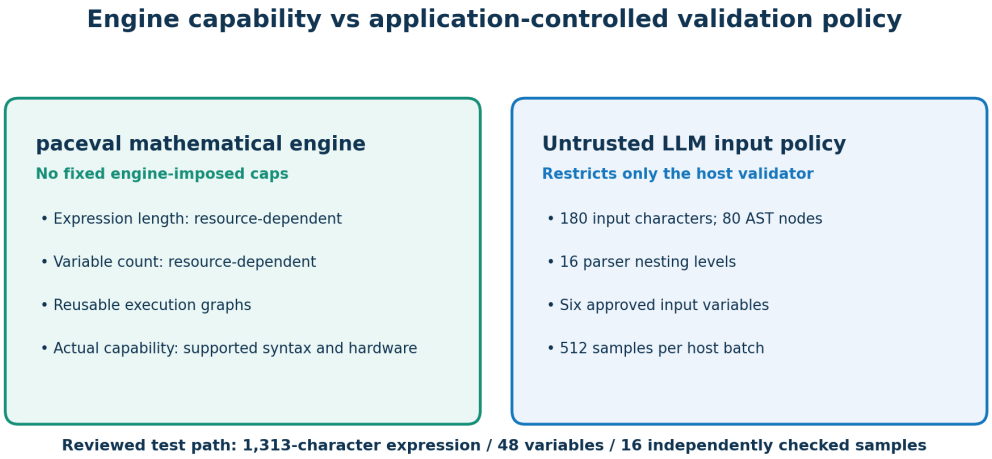


Figure 2. Official engine-level capability is distinguished from the stricter application validation policy for untrusted LLM formula proposals.

Scope	Exactly what is controlled or documented
paceval engine	Vendor documents no fixed function-length or variable-count caps; practical limits depend on resources and supported syntax.
Untrusted LLM interface	At most 180 characters, 80 AST nodes, 16 levels, six named variables and 512 samples per host batch.
Permitted untrusted mathematics	Arithmetic, integer powers 2/3, `sin, cos, exp, ln, sqrt, tanh`; numeric-domain checking for selected cases.
Reviewed model example	1,313 characters, 48 explicitly defined variables, and 16 independently validated input sets.

The untrusted parser rejects unknown operators/functions, unsafe characters, excessive input length and sampled-domain violations before invoking paceval. The reviewed-model path instead requires an **exact match to a host-owned, versioned mathematical definition**. It does not accept arbitrary caller-selected formulas as approved.

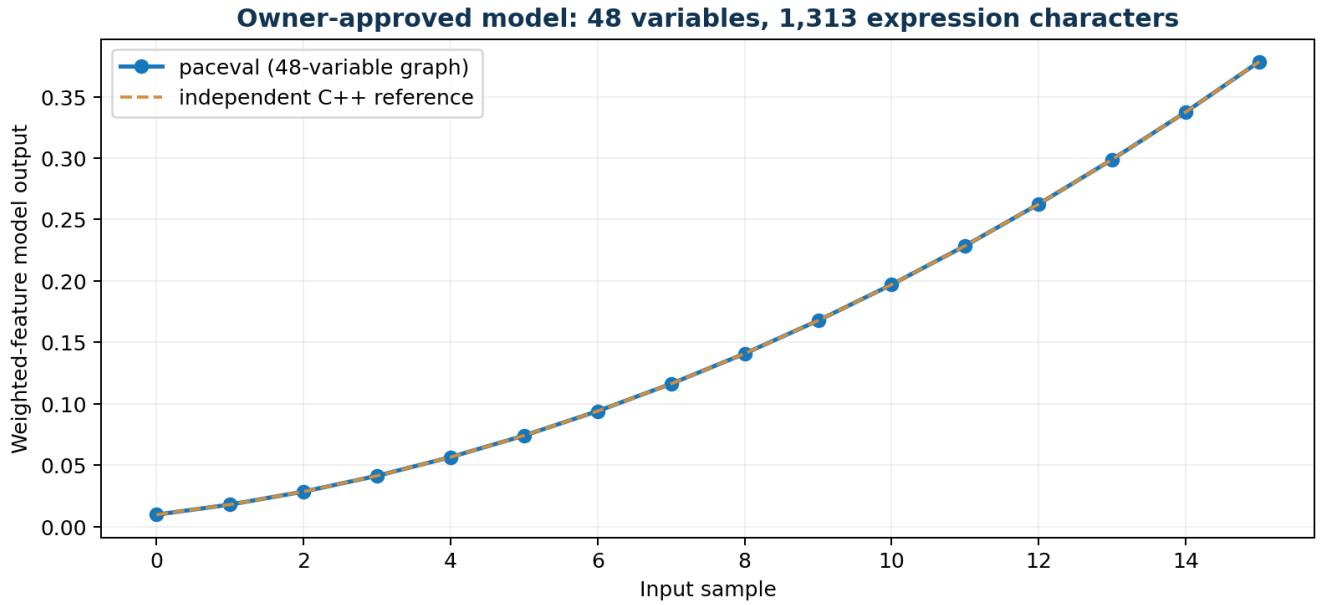
Sources for engine capability: **paceval For Developers** and **Product Brief v4.25** (official vendor documentation). The confirmed 48-variable experiment is an independent, finite runtime test, not proof of unlimited practical hardware resources.

03 / REVIEWED MATHEMATICAL MODELS

Verified 1,313-character expression using 48 variables

To separate paceval engine capability from LLM input policy, this project implements a second, specifically **owner-reviewed mathematical model**, `weighted_features_v1`. It is a transparent weighted sum of linear and quadratic terms over 48 input variables. The model is intended to demonstrate execution-graph reuse and numerical correctness, not a trained neural network or a validated physical system.

1,313	48	16 / 16
mathematical expression characters	distinct input variables	high-precision checks passed



Additional scientific figure. Original paceval output and independently evaluated C++ reference agree for 16 input sets using the same reviewed 48-variable graph.

Approval and independent validation

The exact owner-approved specification is distributed as **examples/approved_48_variable_model.txt**. The C++ host reconstructs its canonical expression and rejects any nonmatching manifest before graph creation. A new 110-decimal Python reference independently recalculates the mathematical sum. All 16 recorded paceval results pass the fixed `3e-11 × (1 + |reference|)` tolerance; a tampered manifest is rejected.

A deliberate negative test submits the same long expression to the **untrusted** interface, which still rejects it under the 180-character host input policy. The two policies therefore coexist without granting untrusted tool callers a bypass.

```
./build/llm_math --approved-model examples/approved_48_variable_model.txt   --approved-output build/
approved_large_model_results.csv
python3 src/validate_approved_model.py --binary build/llm_math   --manifest examples/
approved_48_variable_model.txt
```

These expression and variable counts describe this one demonstrated case, not capacity ceilings of paceval. Generalizing to other approved models requires a separately reviewed formula specification and application-specific resource policy.

03 / DETERMINISTIC STUDY DATASET

200 cases: permitted expressions and deliberate failures

The released corpus is generated deterministically, not independently sampled from a deployed LLM. It comprises **160 permitted variants** (10 mathematical families × 16 coefficient variants) and **40 adversarial or ill-formed variants**. Every permitted expression is applied to 16 fixed parameter samples, yielding exactly 2,560 regular evaluations.

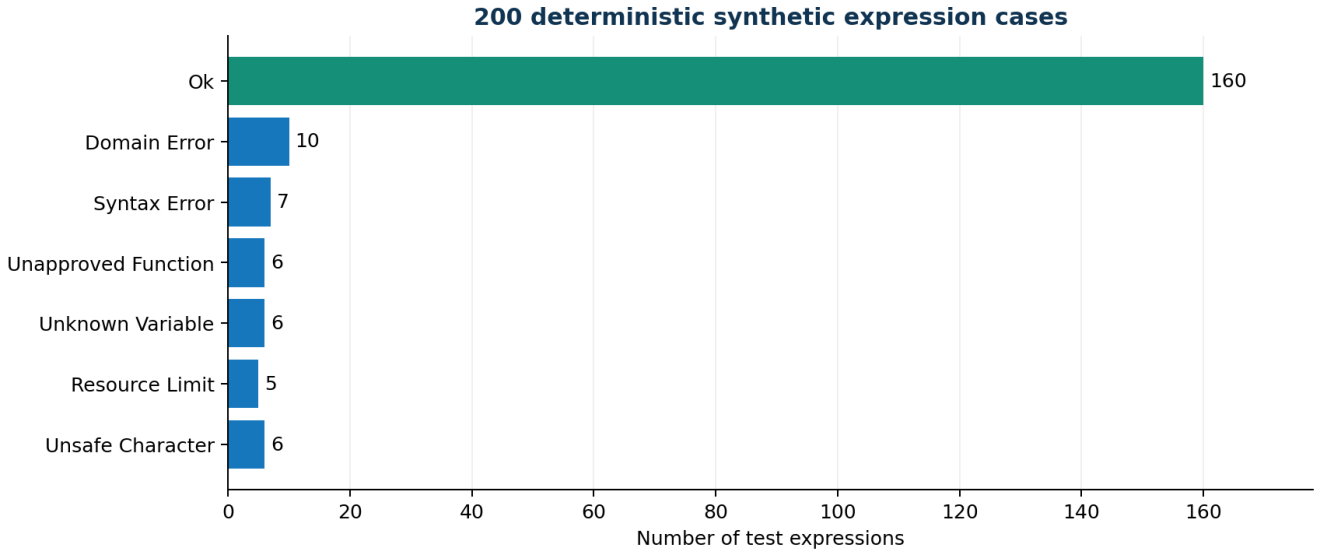


Figure 3. Acceptance and rejection by documented category, directly generated from the 200-case source CSV.

Selected mathematical families

Damped trigonometric response; sums of sine and cosine terms; rational forms with protected denominators; square-root response; logarithmic response; hyperbolic tangent; polynomial/rational blends. Coefficients vary while the grammar and validation policy remain fixed.

Representative invalid input	Expected rejection
`system(x)`	UNAPPROVED_FUNCTION
`1/(t-t)`	DOMAIN_ERROR
`z+t`	UNKNOWN_VARIABLE
`x;system(1)`	UNSAFE_CHARACTER

Complete reference inputs: **examples/expressions.csv** and **examples/sweep.csv**. Results: **examples/evaluation.csv**.

04 / C++ AND PACEVAL IMPLEMENTATION

Real SDK graph construction and reusable evaluation

The application compiles **src/llm_math.cpp** together with the supplied original **src/paceval_main.cpp**, and includes **src/paceval_main.h** directly. The external **libpaceval_linux_sharedLIB.so** is linked at build time and is intentionally not distributed in the archive.

API operation	Use in the code
`paceval_InitializeLibrary`	Initialize the vendor runtime at process entry for mathematical execution.
`paceval_CreateComputation`	Create persistent handles for permitted expressions and TINC-enabled samples.
`paceval_dGetComputationResultExt`	Evaluate batches of input vectors using a prepared expression handle.
`paceval_dGetComputationResult`	Obtain sampled value and TINC lower/upper intervals when interval mode is enabled.
`paceval_GetIsError`	Check graph-level runtime errors after creation and evaluation.
`paceval_DeleteComputation / paceval_FreeLibrary`	Release graph handles and free the library using RAII cleanup.

```
# include "paceval_main.h"    // the original vendor API
# CMake: add_executable(llm_math llm_math.cpp paceval_main.cpp)
# Link the independently supplied Linux runtime as an imported library.
```

Executable graph count and lifetime

Each accepted corpus expression is constructed once and reused over its 16 input samples. The corpus creates 160 regular handles and 10 TINC-enabled handles. The separate reviewed-model test creates an additional 48-variable handle. Additionally, the `--stream` mode keeps the runtime and a bounded graph cache alive while a sequence of validated agent tool requests arrives. The new contract test confirms reuse of two approved expression handles across separate requests. C++ RAII cleanup releases all handles.

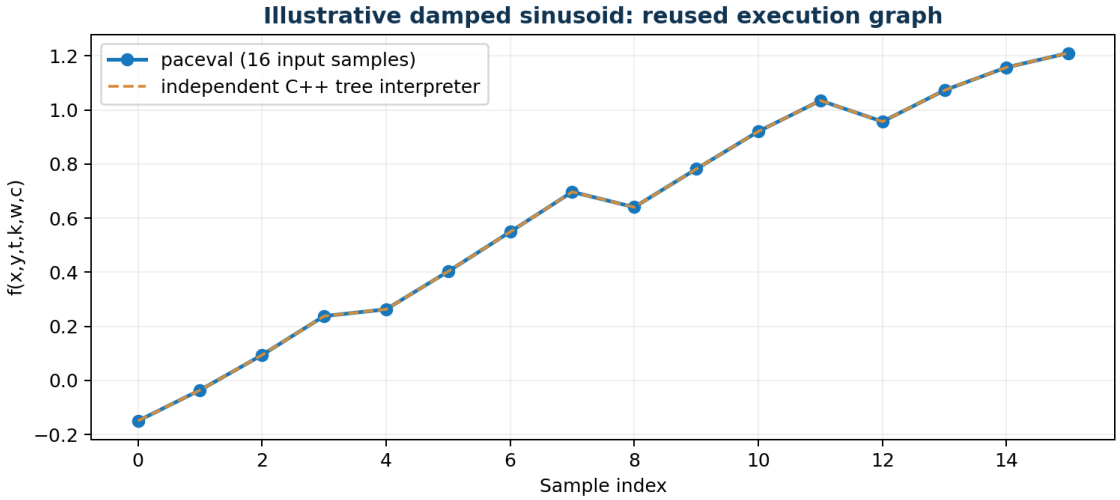


Figure 4. One mathematical graph evaluated across the fixed input sweep, compared to the independent C++ interpreter.

05 / INDEPENDENTLY CHECKED NUMERICAL RESULTS

Agreement between paceval, the C++ AST and 110-digit arithmetic

The regular numerical path was compared to an independently written C++ AST interpreter. A second validation script uses Python's own AST, a restricted function map and `mpmath` with **110 decimal digits** of working precision. That script reads the recorded double-precision input values and re-evaluates every accepted mathematical expression.

2,560	0	4.0×10^{-16}
high-precision comparisons completed	tolerance violations	maximum absolute difference (approximately)

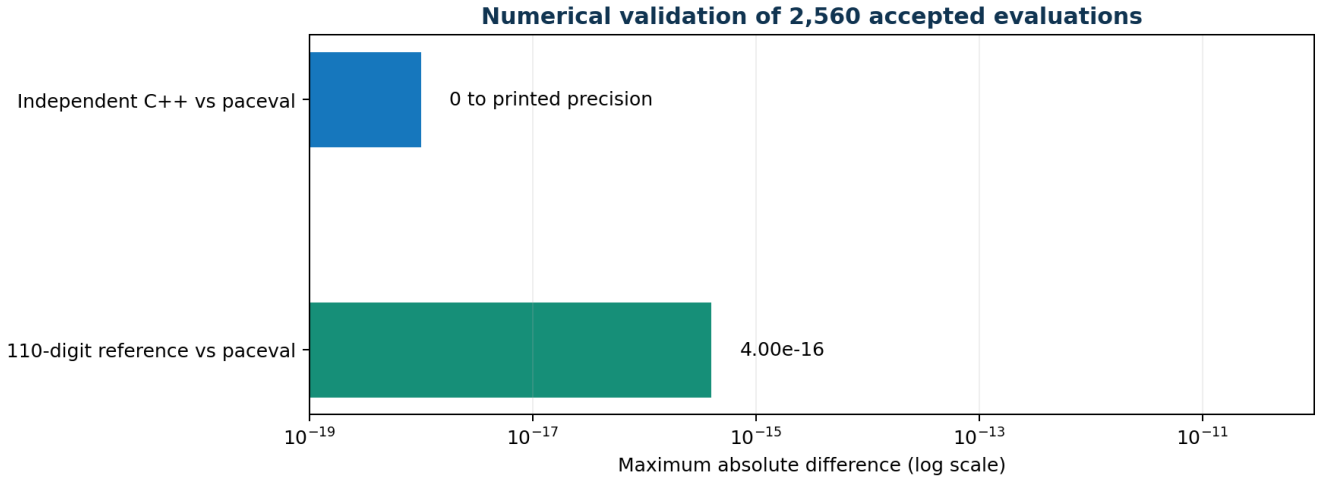


Figure 5. Maximum absolute differences on a logarithmic scale; these are numerical comparisons, not speed measurements.

Interpretation

The predefined acceptance tolerance is `3e-11 × (1 + |reference|)`. All accepted values satisfy it. The small observed differences are consistent with floating-point evaluation differences at the tested magnitudes, but are not proof that every possible formula is well-conditioned.

All error outcomes from invalid inputs are also checked against the predefined expected statuses. Seven automated CTest targets cover parser rules, original-runtime evaluation, deliberate rejection, persistent tool-contract reuse, approved-model calculation and its independent authorization/numerical audit. Runs compiled with GCC and Clang, including sanitizer builds, additionally test the integration and memory diagnostics.

The published 200-case corpus is constructed from ten reproducible formula families. The new contract tests complement it with simulated agent requests that exercise model identity, declared units, value ranges and graph reuse.

06 / TINC INTERVAL INVESTIGATION

Sampled numerical intervals: 30 independently included references

For each of the 10 expression families, one additional computation handle is created with TINC enabled. The demonstrator records the value and reported lower and upper bounds for three fixed input samples (indices 0, 7 and 15). Independent high-precision evaluation tests whether the corresponding numerical reference lies within the reported interval.

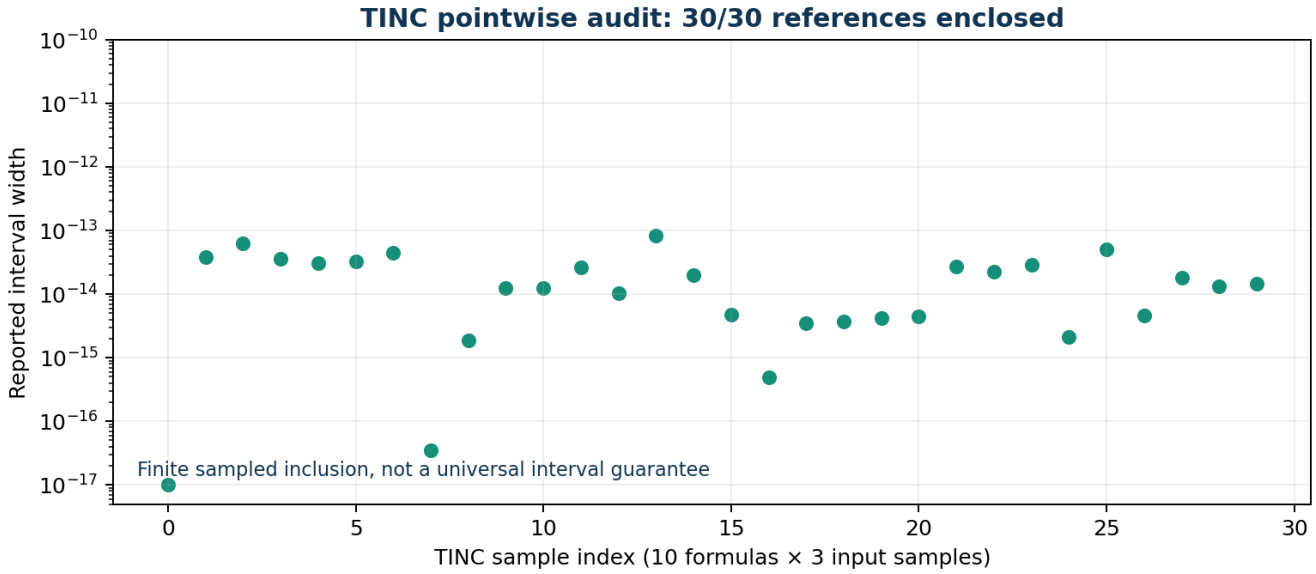


Figure 6. Width of the 30 vendor-reported intervals. Each checked exact reference lies in its recorded interval for the selected inputs.

Audited feature	Observed result
TINC-enabled computation handles	10
Fixed samples per mathematical family	3
Independent arithmetic precision	110 decimal digits
Sampled TINC containment	30 / 30
Evaluation scope	Published 30-point TINC reference test

These recorded TINC results are the 30 fixed input cases in this numerical audit. All 30 include their independently computed high-precision references.

Reproducibility inputs: **examples/tinc_samples.csv**. Independent audit: **src/high_precision_reference.py**; machine-readable summary: **examples/validation_summary.json**.

07 / AGENT MODEL CONTRACTS AND PERSISTENT REUSE

A verified LLM-oriented request workflow

In a technical agent workflow, an LLM may propose both a formula and named parameters. The host independently selects a reviewed model contract: this pilot defines two versioned, exact mathematical formula contracts, `damped_signal_v1` and `harmonic_signal_v1`. Each has a declared unit signature: time `t` in seconds, rate/frequency `k` and `w` in inverse seconds, and the remaining inputs dimensionless.

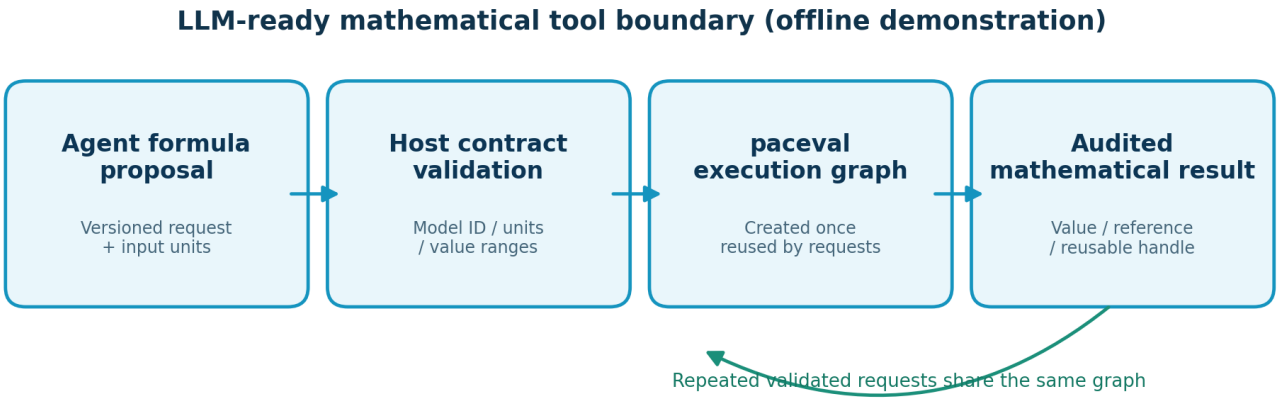


Figure 7. The added host-level contract gate preserves approved model semantics and keeps paceval computation handles alive across repeated structured requests.

Workflow check	Observed result
Structured proposals submitted	9
Approved mathematical requests	5, all matching independent C++ references
Incorrect or unknown requests rejected	4: units, model mismatch, domain and unknown contract
Persistent graph reuse	Confirmed for both approved model contracts
Implementation	`src/llm_contract_adapter.py`, `--stream` mode of `src/llm_math.cpp`, and `src/validate_llm_workflow.py`

Correct mathematical responsibility split

paceval evaluates approved explicit expressions and reuses persistent computation graphs. The host enforces exact model identity, dimension-label agreement, bounded parameter values and independent numerical cross-checking. The LLM itself does not control executable code, library initialization, resource limits or graph selection.

The published structured tool definition and sample results are **examples/llm_contract_schema.json**, **examples/llm_proposals.jsonl**, and **examples/llm_responses.jsonl**. These are deterministic sample requests, not measured generations from a deployed language model. Contract matching is exact and versioned; it is not a general symbolic formula-equivalence or units inference engine.

08 / REPRODUCING THE EXPERIMENT

Build, test and inspect the offline tool interface

The only external paceval runtime requirement is a compatible local Linux shared library. The GitHub package contains the SDK translation unit and header but intentionally excludes the proprietary `.so`. After extracting the project, run:

```
export PACEVAL_LIB=/absolute/path/to/libpaceval_linux_sharedLIB.so
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME="$PACEVAL_LIB"
cmake --build build -j
ctest --test-dir build --output-on-failure
./build/llm_math --cases examples/expressions.csv \
  --sweep examples/sweep.csv --output build/evaluation.csv \
  --tinc build/tinc_samples.csv --batch 37
python3 src/high_precision_reference.py --cases examples/expressions.csv \
  --sweep examples/sweep.csv --results build/evaluation.csv \
  --tinc build/tinc_samples.csv
```

Structured, limited tool-call example

```
{
  "request_id":"demo", "expression":"exp(-k*t)*sin(w*t)+c",
  "variables":{"x":0.2,"y":-0.1,"t":0.5,"k":0.3,"w":1.2,"c":0.1}
}
```

The accompanying **src/agent_tool_adapter.py** reads JSON Lines, checks the requested fields and finite values, invokes the compiled C++ binary with argument lists (not a shell), and writes structured responses. An additional **src/llm_contract_adapter.py** implements approved model IDs, explicit unit signatures, input ranges and a persistent local C++ process for multiple requests.

> The contract-workflow samples are **examples/llm_proposals.jsonl** and **examples/llm_responses.jsonl**. The original input/output examples are **examples/tool_schema.json**, **examples/tool_calls.jsonl** and **examples/tool_responses.jsonl**. Build logic is in **src/CMakeLists.txt**. Full run outputs are distributed in **examples/**.

Reproducibility checks carried out

GCC and Clang builds completed successfully. AddressSanitizer and UndefinedBehaviorSanitizer produced no findings in the tested cases. Full untrusted-corpus CSVs generated using batch sizes 1, 37 and 512 were byte-identical. An independent 110-decimal audit also passed all 16 values of the owner-reviewed 48-variable model. The high-precision audit was rerun on the published outputs, and a fresh-extraction build of the final ZIP is part of the release check.

09 / CONCLUSIONS AND REFERENCES

Feasibility shown, with explicit limits and a practical extension path

Confirmed result: The original paceval SDK can evaluate a selected corpus of mathematically validated, reusable LLM-like expressions from a restricted offline tool path. The host rejects all 40 deliberately invalid corpus cases, evaluates the 160 accepted expressions on 16 input sets, and independently checks the numerical results. The added model-contract workflow confirms semantic policy rejections and persistent graph reuse. A separate owner-approved 1,313-character / 48-variable model also passes 16 high-precision checks, demonstrating that the untrusted policy is not an engine limit.

What remains outside the demonstrated claim

- The proposal corpus and nine structured agent requests are deterministic test fixtures; live-model integration remains a separate step.
- The fixed 180-character restriction applies only to untrusted host input, not paceval. Contract-unit signatures are checked; general symbolic dimensional analysis and whole-domain mathematical certification are future extensions.
- Deployment concerns such as authentication, process isolation, quotas and MCP transport remain with the host environment.
- This paper reports correctness and integration evidence, not comparative hardware performance.

Useful next research extensions

Integrate a real model proposal source using strict structured outputs; expand the adversarial corpus and independent curation; add unit-aware expression typing and stronger domain proof; isolate evaluation in a restricted process; implement a proper MCP tool server with authentication, quotas and logging; and audit the end-to-end task quality in a documented engineering workflow.

External scientific and technical sources

[1] paceval UG. *paceval C/C++ API v4.25*. paceval.com/api/; *For Developers*, paceval.com/developers/; *Product Brief*, paceval.com/product-brief/.

[2] Model Context Protocol. *Tools specification, 2026-07-28*. modelcontextprotocol.io/specification/2026-07-28/server/tools

[3] OWASP. *LLM Prompt Injection Prevention Cheat Sheet*. OWASP Cheat Sheet Series

[4] NIST. *Artificial Intelligence Risk Management Framework: Generative AI Profile (AI 600-1), 2024*. NIST publication

[5] paceval supplied research proposal. *WP_003: Evaluating LLM-Generated Mathematical Expressions*, November 2026 (scope and initial hypothesis; not included here as an experimental result).

The validated artifact is contained in this repository: **README.md**, **src/**, **examples/**, **figures/** and this current whitepaper. All measured numerical findings refer to the included synthetic corpus and owner-approved 48-variable test case. The supplied original vendor SDK source files retain their copyright and licensing statements.